

VAYU-NET

Project Report

Multi-agent urban air quality intelligence over India's CPCB CAAQMS network
Delhi · Mumbai · Bengaluru

This report states what VAYU-NET is, what it currently does on live government data, what has been proven about it, and what is not yet true. It is written to be read by someone deciding whether to pilot the platform — so the limitations are stated as plainly as the capabilities.

field	value
Generated	2026-08-29
Status	Pilot-ready. Verified on live data. Not yet handover-complete — see §8
Companion documents	VAYU-NET_System_Design.pdf · VAYU-NET_Test_Plan.pdf
Frontend	https://vayu-net-ten.vercel.app
API	https://vayu-net-api-ver-tex.vercel.app
Data store	Supabase Postgres (ap-south-1)
Training data	supabase readings_hourly (live)
Data window	2025-12-25 05:00:00 -> 2026-08-29 18:00:00

What makes this different from an AQI dashboard. VAYU-NET does not stop at showing a number. It attributes an episode to a likely source class with a confidence score, ranks which ward an officer should act on first, cites the statute that applies, and generates the evidence pack to support it. The product is prioritisation and evidence assembly — never proof of liability, and the interface says so.

Contents

§	section
1	What the platform does
2	Architecture and the agent chain
3	The data: coverage, freshness, and one honest caveat
4	What is proven — verification results
5	The ward estimator: the defect that mattered most
6	Forecasting: measured honestly, and currently negative
7	Security posture
8	What is not yet true — gaps before agency handover
9	Engineering history: defects that shipped once
10	Recommended next investments

1 · What the platform does

VAYU-NET ingests hourly readings from India's CPCB CAAQMS network for three cities, detects pollution episodes against each station's own baseline, attributes each episode to a likely source class, ranks wards for enforcement attention, and issues public health advisories in five languages. It runs unattended on an hourly schedule.

capability	what it produces	state
Live measurement	Hourly PM2.5/PM10 per sensor, CPCB National AQI, band and health guidance. Freshness read from the database clock, not a build artefact.	verified live
Ward-level view	Every ward resolves to measured, estimated or unavailable — a ward with no sensor within 8 km is shown no number at all.	verified live
Episode detection	Per-station z-score against a rolling baseline, plus severe-threshold crossings. 1,588 attributed events in the store.	verified live
Source attribution	Evidence-weighted likelihood across traffic, biomass/fireworks, construction dust, industrial and secondary regional, with a confidence score and the signals behind it.	verified live
Enforcement ranking	Wards ranked by severity x confidence x persistence x vulnerability, with a statutory citation and a one-page evidence-pack PDF.	verified live
Advisories	Health text per ward, per risk group, in English, Hindi, Marathi, Kannada and Tamil. Provenance (measured / estimated / forecast) is always stated.	verified live
Forecasting	LightGBM, five horizons from 6h to 72h, per station. Now trains on the live store. Currently does not beat a persistence baseline — see §6.	not yet useful

2 · Architecture and the agent chain

Five agents run in sequence, each reading and writing the same Postgres store. The deployed API serves exclusively from that store — there is no bundled snapshot behind the live view, which is what makes the freshness banner trustworthy.

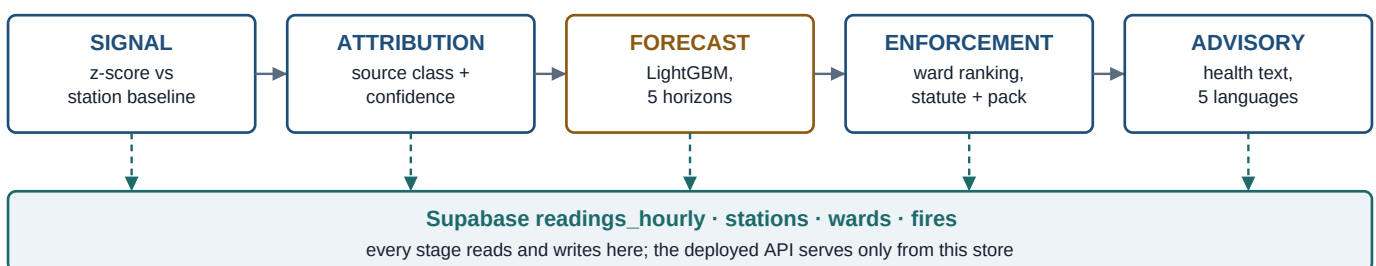


Figure 1 — the agent chain. The forecast stage is amber: it is the one stage whose output is not currently better than a trivial baseline.

layer	technology	note
Ingest	data.gov.in CPCB feed, hourly	Serves only the current hour — a missed hour is gone permanently
Schedule	Supabase pg_cron, 5 * * * *	GitHub Actions is the backup path, not the primary
Store	Supabase Postgres, ap-south-1	9 tables; secrets resolve env-first then from ops_config
Agents	Python, LightGBM, pandas	Chain: signal, attribution, forecast, enforcement, advisory
API	FastAPI on Vercel	60s function budget; all row counts are capped
Frontend	React + Vite + MapLibre	Separate Vercel project, same repo
Satellite	NASA FIRMS	17,063 fire detections; fails soft by design
Language	Claude, budget-capped	Falls back to a CPCB template if the model is unavailable

3 · The data: coverage, freshness, and one honest caveat

city	sensors	ward-mapped	live enforcement actions
Delhi	46	46	1 (plus 10 from the archived episode)
Mumbai	29	28	0 — air is 21-34 ug/m3, correctly no action
Bengaluru	14	13	0 — air is 21-34 ug/m3, correctly no action

29,193 hourly readings across 90 sensors span 2025-12-25 to the current hour, of which roughly 15,200 are from the live era. The store carries both a severe December episode and the current monsoon period — a range that matters for \$6.

Data integrity, checked directly against the store: no negative concentrations, no readings dated in the future, no duplicate station identities, station/hour keys unique across all 29,193 rows, and all attribution confidences within range. 2,093 rows carry no PM value at all — sensor downtime, correctly rendered as no-data rather than as zero.

Upstream caveat, disclosed rather than hidden. 475 readings (1.6%, across 34 stations) report PM2.5 greater than PM10, which is physically impossible since PM2.5 is a subset of PM10. This is instrument disagreement in the CPCB source data, not an ingest defect. It does not change any reported AQI band, because the CPCB index takes the maximum of the pollutant sub-indices rather than combining them. It is stated here so an agency reviewer meets it in the documentation rather than discovering it in the data.

Cadence is the binding constraint on data quality. data.gov.in serves only the current hour and there is no backfill, so every missed hour is permanently lost. Under the GitHub free scheduler, capture degraded to 1-3 hours per day. Moving the schedule to Supabase pg_cron restored it to 15-20. Uptime of that job is, directly, the rate at which the platform's historical asset accumulates.

4 · What is proven — verification results

Three suites run against the platform. The distinction between them is not cosmetic: one of them proves things about production, and two do not.

suite	what it actually tests	result
verify_live.py	18 assertions. Drives a local TestClient against the remote database — so it validates this working tree, not the deployed function.	17 pass, 1 warns (RLS parity, open by choice)
verify_edge_cases.py	18 input-validation and abuse cases. Runs against either the local app or a deployed URL, with identical assertions — so local/production drift is visible rather than inferred.	18/18 local 18/18 production
verify_ward_estimate.mjs	12 estimator assertions against live station data, including the measured / estimated / unavailable boundaries.	12/12

A green suite is not a statement about production. On 2026-08-29 verify_live.py reported ALL PASSED while the deployed API answered "AQI -50 (Severe) measured now" to any caller who set that query parameter. It drives a local TestClient, so it was describing the working tree. That is precisely why the edge-case suite takes a target URL, and why the accuracy claims below were re-checked against the deployed endpoint rather than the local one.

Measurement accuracy, verified against production. All 46 Delhi station values served by the deployed API match the database exactly — zero drift — and every AQI and band matches an independent recomputation from the published CPCB breakpoint tables. Zero mismatches.

5 · The ward estimator: the defect that mattered most

Most Delhi wards have no sensor of their own. The citizen view filled that gap with the city arithmetic mean — for 251 of 289 wards — and rendered it in 46px type under the ward's own name, styled exactly like a measured reading. On the morning this was found, those wards displayed "AQI 130 · Moderate · reduce prolonged outdoor exertion" while the sensor at Anand Vihar measured 448, Severe. The map was right and the advisory was wrong, from the same store, at the same moment.

MEASURED sensor inside the ward worst sensor wins Delhi: 40 wards	ESTIMATED no sensor, one within 8 km inverse-distance weighted Delhi: 247 wards	UNAVAILABLE nearest sensor > 8 km NO number is shown Delhi: 3 wards
---	---	---

Figure 2 — three outcomes, never blended. Before this, 251 of Delhi's 289 wards showed the city arithmetic mean in 46px type, styled exactly like a real reading.

The replacement is inverse-distance weighting over sensors within 8 km, with three explicit outcomes and no blending between them. A ward containing several sensors reports the **worst**, not the average — an advisory is a health instruction for everyone in the ward, and averaging away the dangerous corner of it is the same error in miniature. Where a nearby sensor sits in a worse band than the estimate, that sensor is named in red rather than smoothed away; 76 Delhi wards currently carry such a warning.

Delhi now resolves 40 measured, 247 estimated and 3 refused, with **117 distinct values** in place of one repeated number. The API and the advisory agent both carry a `basis` field, and the client refuses any server advisory that does not echo it — so an API predating the field cannot describe an interpolated number as "measured now".

What testing cannot establish here. An interpolated ward AQI has no reference value to compare against — that is precisely why the ward has no sensor. The tested property is the *label*, not the number: that the outcome is correctly classified, correctly masked beyond 8 km, and never described as measured.

6 · Forecasting: measured honestly, and currently negative

Until 2026-08-29 the forecast model trained on a bundled snapshot frozen at 2025-12-25 to 2026-01-01. The nightly retrain re-fit the same seven days of December every night and never saw a single row the platform had collected. Nothing failed and the job stayed green — the model was simply predicting monsoon air from a severe winter episode. It now trains on the live store, which already contained that December window and everything since.

Two further defects surfaced with it. Lag features were **positional rather than temporal**: 11% of consecutive readings are not an hour apart, so a plain shift presented a reading up to 5,438 hours old as "one hour ago". Features are now built on a complete hourly grid. And the 36-hour test window was **narrower than the 48h and 72h horizons**, so both were fit, shipped and reported zero test rows beside validated horizons. The window is now seven days and all five horizons are backtested.

horizon	test rows	persistence RMSE	model RMSE	verdict
6h	1823	7.49	16.14	persistence much better
12h	1222	11.27	15.62	persistence better
24h	1208	17.80	17.58	tie
48h	798	25.72	27.46	persistence better
72h	544	29.06	29.06	tie

The previously published figure of +35.7% versus persistence was not wrong — it was measured on the wrong regime. During December's episode PM2.5 swings violently and a persistence baseline scores an RMSE near 100, so a model beats it easily. On calm monsoon air persistence scores 7.49 and is very hard to beat. Four training strategies were compared — full history, live-era only, and recency half-lives of 30 and 7 days — and persistence won three of five horizons under every one of them. This is a property of the conditions, not an artefact of the training set.

Full history is retained as the training default regardless, because December is the only severe-episode data that exists and severe episodes are what the platform is for. The verification suite asserts the training provenance — so a silent fall back to the stale snapshot now fails the build — but it deliberately does **not** assert that the model beats persistence, because an assertion like that

only encourages quoting a number that is not currently true.

Recommendation. Do not present the forecast as an accuracy claim in a pilot evaluation. Present the measurement, attribution and enforcement layers, which are verified. Re-measure the forecast during stubble-burning or winter season — the regime in which persistence fails and a model can earn its place.

7 · Security posture

control	state
Input validation	All public query parameters are validated and bounded. Eight defects reachable by any caller — including an out-of-range AQI returned as measured health guidance, and three paths that answered HTTP 500 — were fixed and verified on production.
SQL injection	Every user-supplied string reaches Postgres as a bound parameter. Probed with three payload classes; all inert, tables intact.
Row-count limits	Capped at 5,000. An unbounded LIMIT was the cheapest way to exhaust the 60-second serverless budget on an unauthenticated endpoint.
Ingest authentication	Token-protected; rejects an absent, wrong or empty token with 401. The token resolves environment-first then from ops_config, so both ends of the scheduled call agree by construction.
Secret storage	ops_config has RLS enabled with no policy and no public grants; the suite asserts it. Supabase serves every public-schema table over PostgREST, so this matters.
RLS parity	OPEN. fires and llm_spend have RLS disabled while the anon role holds INSERT, UPDATE, DELETE and TRUNCATE. The other seven tables have RLS on, which is the only thing neutralising identical grants. fires feeds the attribution agent, so this is a write path into evidence inputs. Latent only because the anon key is not published — the frontend talks to the API and never to Supabase. Two SQL statements close it. Until they run the suite reports this as a warning rather than a failure, so a deliberately deferred item does not take the hourly ingest down with it.
Authentication	NONE. Every endpoint answers unauthenticated, CORS is *, and POST /replay/run both mutates the store and bills the language-model budget without a credential. This is the single largest gap before agency handover.

8 · What is not yet true — gaps before agency handover

In build order. None of these are defects; they are unbuilt capability, and each one blocks a specific operational reality.

#	gap	what it blocks
1	No authentication	No agency can use a system where any URL holder reads everything and can trigger a billed, state-mutating run. Everything below depends on this.
2	No multi-tenancy	One board cannot be prevented from seeing another's wards, so the platform cannot be sold to two customers at once.
3	No alerting	A stalled feed or failed cron is noticed by a person looking at the banner. There is no notification path to an officer when a ward crosses a threshold.
4	No action workflow	An action can be ranked and evidenced but not assigned, tracked or closed — so the platform informs enforcement without participating in it.
5	Forecast unproven in-regime	See §6. Validated, and currently negative on calm air. Needs re-measurement across a high-variance season before it can be quoted.
6	Free-tier infrastructure	No SLA, no redundancy, no support path. Acceptable for a pilot, not for a service an agency depends on.
7	RLS parity	Two SQL statements, deliberately left open. See §7.

9 · Engineering history: defects that shipped once

Each of these reached a running system and each now has an assertion guarding it. They are listed because the pattern is more useful than the individual fixes: in almost every case the system stayed green while telling a false story.

defect	why it was invisible
Duplicate station identity	The archive keyed sensors site_NNN, the live feed by name. Every sensor existed twice and the two sets were exactly disjoint — every ward-mapped row frozen, every live row unmapped. Enforcement joins on ward, so 300/300 recent Delhi events were structurally unreachable by it. 163 rows merged to 90 real sensors.
City mean shown as a ward reading	Correct data, correct map, wrong number in the one place a citizen reads. See §5.
Forecast trained on a frozen archive	The nightly retrain succeeded every night and never saw live data. See §6.
Negative and out-of-range AQI banded Severe	aqi=-50 returned "AQI -50 (Severe) measured now" — health guidance over an impossible number, from a query parameter any caller could set.

defect	why it was invisible
band() returned Severe for NaN	Every comparison against NaN is false, so no-data sensors fell through the band loop to the default and were painted blood red.
Timezone skew	Readings are IST wall-clock, the database clock is UTC. Every window comparison was skewed 5h30m, putting the newest reading three hours in the future.
Freshness from a build artefact	The banner read a bundled file 12 days stale while the store was 2 hours old.
Enforcement windowing and floor	One shared pool let December's crisis permanently outrank every current event, so live could never show an action. Separately, severity clipped at 60 ug/m3, so whole cities of actions tied at priority 0.00, each citing a statute over officially-good air.
Secrets lost in the dashboard	A Vercel environment variable applies only at build time and only to the owning project. A value saved against the wrong project is indistinguishable, from inside the function, from one never set. Secrets now resolve from ops_config, which is verifiable from SQL.

10 · Recommended next investments

#	investment	why now
1	Close RLS parity — two SQL statements	Cheapest item on this list by a wide margin, and it is a write path into enforcement evidence. There is no reason to carry it into a pilot.
2	Authentication and multi-tenancy	The gate on every commercial conversation. Until it exists the platform demonstrates well and cannot be deployed.
3	Protect the ingest cadence	A missed hour is unrecoverable. The historical asset — and therefore any future forecast claim — accrues at exactly the rate this job stays up. It deserves monitoring before anything else is built on top of it.
4	Alerting	Turns a dashboard into an operational tool, and is the first feature an officer will ask for after seeing the ranking.
5	Re-measure the forecast in season	Stubble-burning and winter are the regimes the model was built for and the ones where persistence fails. Either it earns its place there or it should be presented as an experiment.
6	Action workflow — assign, track, close	Converts evidence assembly into a system of record, which is what makes the platform difficult to displace.

Closing note

VAYU-NET's measurement, attribution and enforcement layers are verified against live government data and behave correctly under adversarial input. Its forecasting layer is now measured honestly and does not currently beat a trivial baseline, which is worth stating plainly rather than discovering during an evaluation. The distance between this and a deployable agency system is authentication, tenancy and operational monitoring — none of it research, all of it build. The differentiator holds: answering which ward, why, and under what statute, with the evidence pack attached.